

Data Structures and Algorithms

GÉZA HORVÁTH

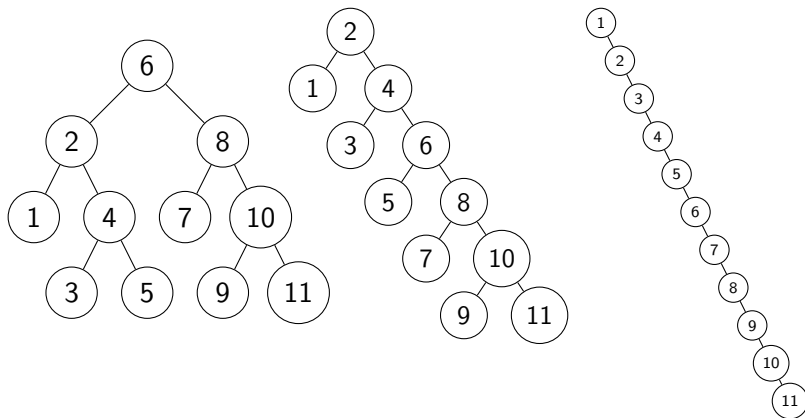
Eighth Lecture

Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 **Red-black trees.**
- 9 B-trees.
- 10 Graphs, traversal, shortest path.
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

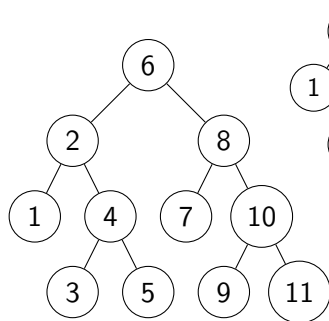
Search trees

How long it takes to find a key?

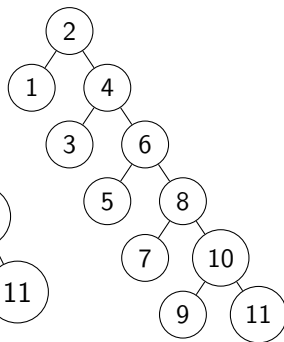


Search trees

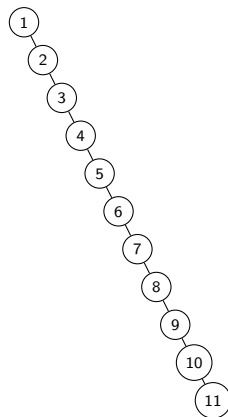
How long it takes to find a key?



Logarithmic!

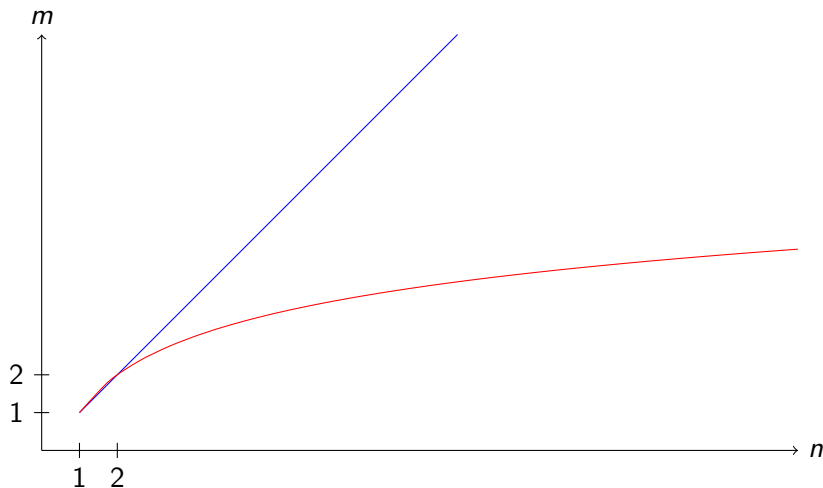


Linear.



Linear.

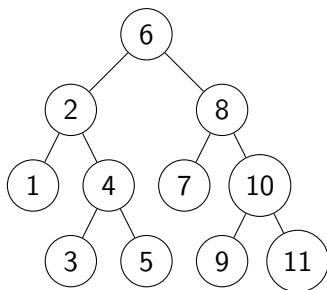
Linear and logarithmic search comparison



n – input size

m – running time

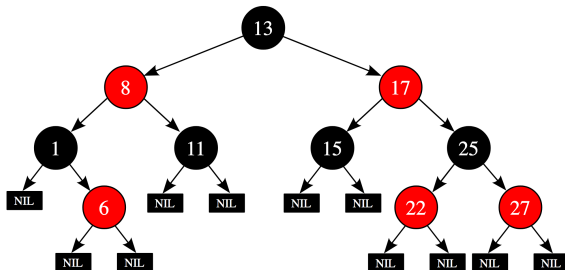
Search trees



- TREE-SEARCH – $\mathcal{O}(\log_2 n)$
- TREE-INSERT – $\mathcal{O}(\log_2 n)$
- TREE-DELETE – $\mathcal{O}(\log_2 n)$

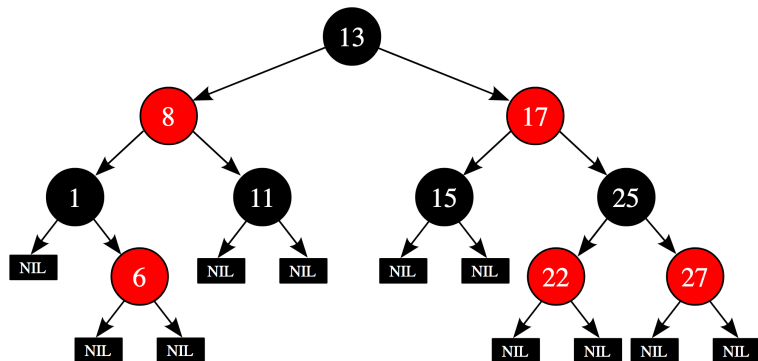
Good case, balanced tree. – **Is there any other solution???**

Red-black tree



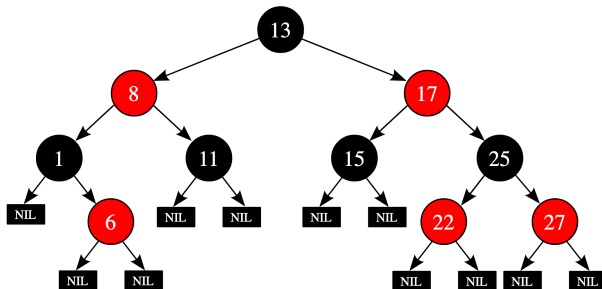
In computer science, a red-black tree is a self-balancing binary search tree. It was invented in 1972. A red-black tree is a binary search tree with one extra bit of storage per node: its color, which can be either RED or BLACK. By constraining the node colors on any simple path from the root to a leaf, red-black trees ensure that no such path is more than twice as long as any other, so that the tree is "**approximately balanced**".

Red-black tree



Each node of the tree now contains the attributes color, key, left, right, and p. If a child or the parent of a node does not exist, the corresponding pointer attribute of the node contains the value NIL.

Red-black tree

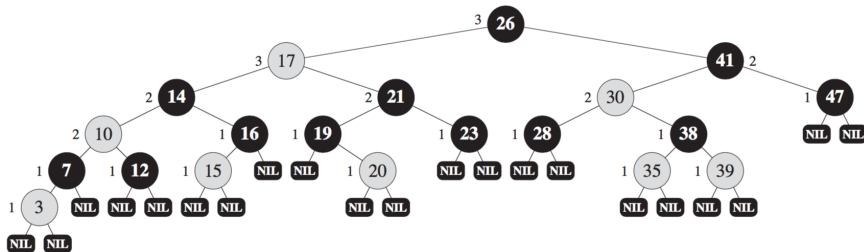
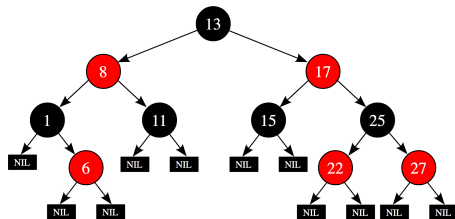


A red-black tree is a self-balancing binary search tree that satisfies the following **red-black properties**:

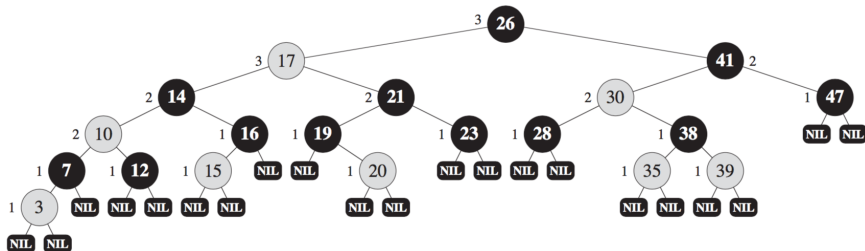
1. Every node is either red or black.
2. The root is black.
3. Every leaf (NIL) is black.
4. If a node is red, then both its children are black.
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes.

Red-black tree

Are red-black trees
balanced???



Red-black tree



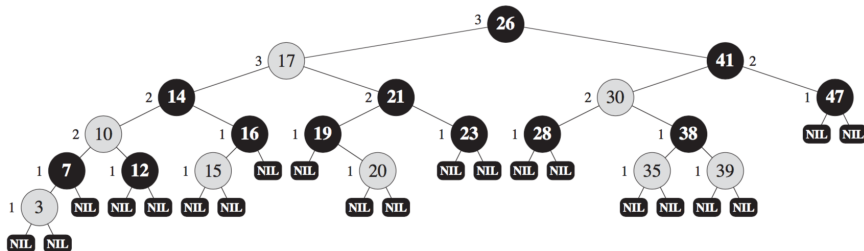
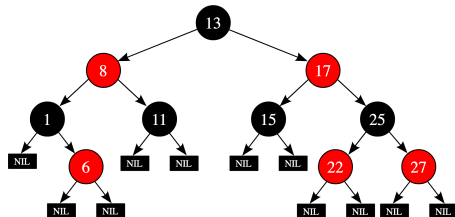
A red-black tree is a self-balancing binary search tree that satisfies the following **red-black properties**:

1. Every node is either red or black.
2. The root is black.
3. Every leaf (NIL) is black.
4. If a node is red, then both its children are black.
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes.

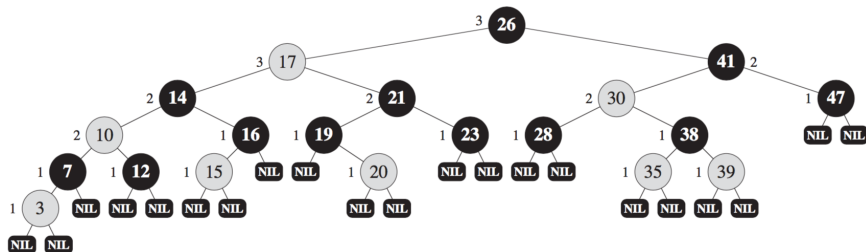
Red-black tree

Are red-black trees
balanced???

Sometimes yes,
sometimes not.



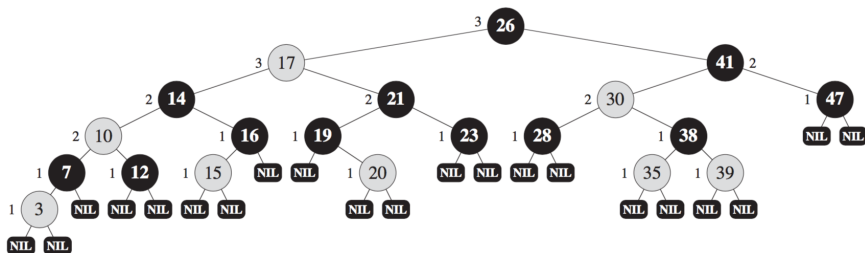
Red-black tree



Lemma

*A red-black tree with n internal nodes has height at most $2 * \log_2(n + 1)$.*

Red-black tree



- TREE-SEARCH – $\mathcal{O}(\log_2 n)$
- TREE-INSERT – $\mathcal{O}(\log_2 n)$
- TREE-DELETE – $\mathcal{O}(\log_2 n)$

Good case, even if it is not – always – balanced tree.

Red-black tree – data structure operations

Implemented data structure operations:

- **create** data structure
- **modify** data structure
 - add elements – $\text{TREE-INSERT} + \text{RECOLOR} + \text{ROTATE}$
 - delete elements – $\text{TREE-DELETE} + \text{RECOLOR} + \text{ROTATE}$
- **access** – TREE-SEARCH

We do not use "REBALANCE" operation, because red-black trees do not have to be balanced. We recolor nodes, and use rotations to restore the red-black properties, instead.

Red-black tree – rotation

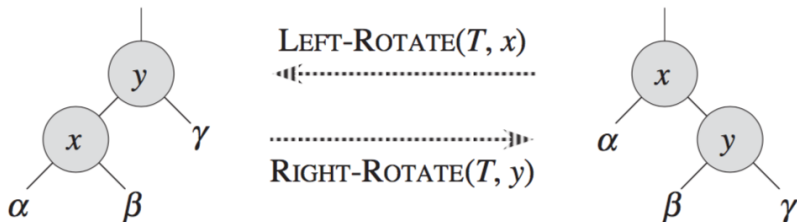
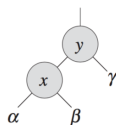


Figure 13.2 The rotation operations on a binary search tree. The operation $\text{LEFT-ROTATE}(T, x)$ transforms the configuration of the two nodes on the right into the configuration on the left by changing a constant number of pointers. The inverse operation $\text{RIGHT-ROTATE}(T, y)$ transforms the configuration on the left into the configuration on the right. The letters α , β , and γ represent arbitrary subtrees. A rotation operation preserves the binary-search-tree property: the keys in α precede $x.\text{key}$, which precedes the keys in β , which precede $y.\text{key}$, which precedes the keys in γ .

Red-black tree – rotation

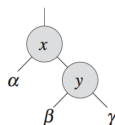


LEFT-ROTATE(T, x)

.....

.....

RIGHT-ROTATE(T, y)



LEFT-ROTATE(T, x)

```

1   $y = x.right$                                 // set y
2   $x.right = y.left$                              // turn y's left subtree into x's right subtree
3  if  $y.left \neq T.nil$ 
4       $y.left.p = x$ 
5   $y.p = x.p$                                     // link x's parent to y
6  if  $x.p == T.nil$ 
7       $T.root = y$ 
8  elseif  $x == x.p.left$ 
9       $x.p.left = y$ 
10 else  $x.p.right = y$ 
11  $y.left = x$                                     // put x on y's left
12  $x.p = y$ 
```

Red-black tree – rotation

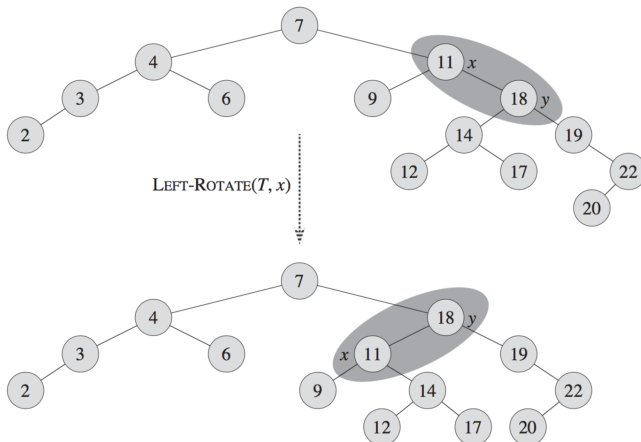


Figure 13.3 An example of how the procedure $\text{LEFT-ROTATE}(T, x)$ modifies a binary search tree. Inorder tree walks of the input tree and the modified tree produce the same listing of key values.

Red-black tree – insertion

Inserted node is always red.

Which of the red-black properties might be violated?

- 1, 3 and 5 can not be violated,
- only 2 or 4 can be violated.

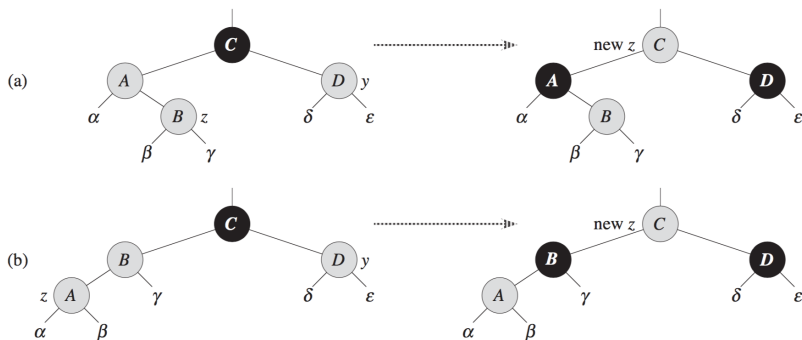
2. The root is black. – When the root is red, simply change it to be black.

4. If a node is red, then both its children are black. – Violating this is the only "serious" problem. We distinguish 3 cases:

- Case 1: z's uncle is red
- Case 2: z's uncle is black and we have triangle
- Case 3: z's uncle is black and we have line

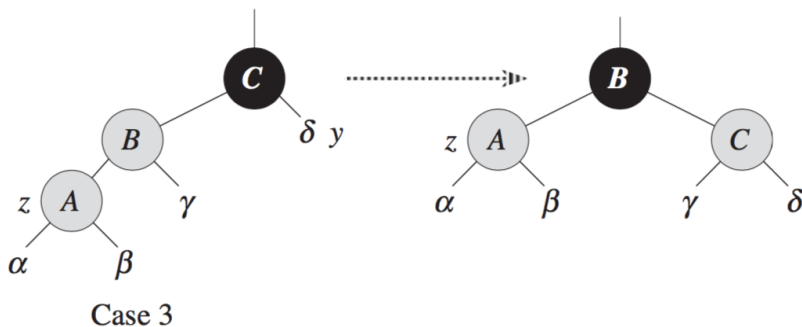
where z is the problematic node.

Red-black tree – insertion – uncle red



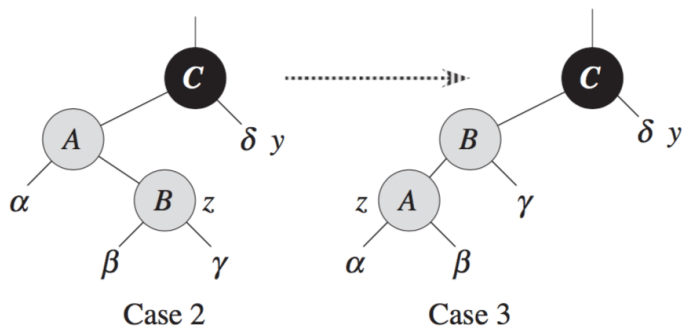
Solution: Recolor parent, grandparent and uncle.

Red-black tree – insertion – line



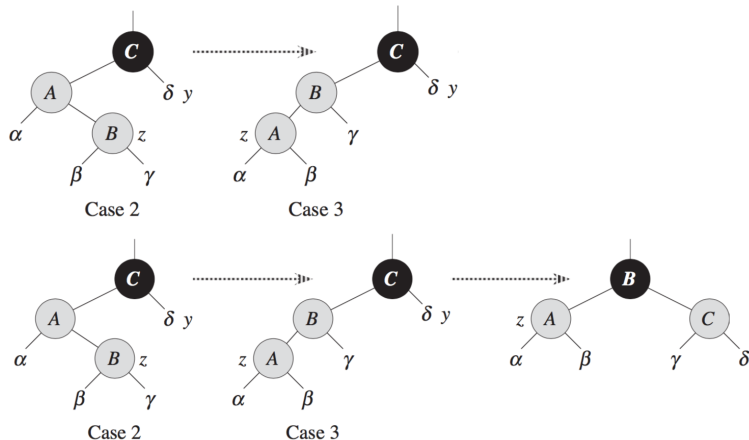
Solution: Rotate and recolor parent and sibling. (Or recolor parent, grandparent and rotate.)

Red-black tree – insertion – triangle



Solution: Rotate.

Red-black tree – insertion – triangle



Red-black tree – insertion – example

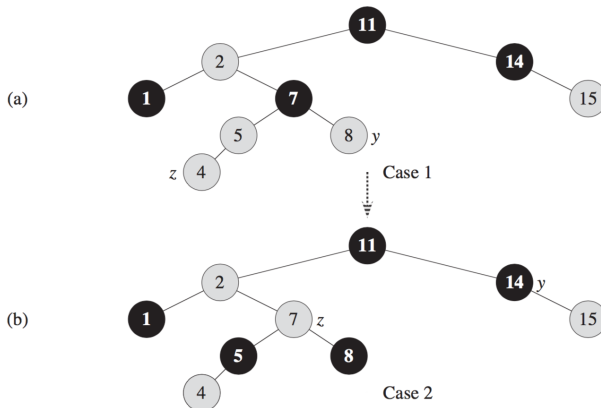


Figure 13.4 The operation of RB-INSERT-FIXUP. (a) A node z after insertion. Because both z and its parent $z.p$ are red, a violation of property 4 occurs. Since z 's uncle y is red, case 1 in the code applies. We recolor nodes and move the pointer z up the tree, resulting in the tree shown in (b). Once again, z and its parent are both red, but z 's uncle y is black. Since z is the right child of $z.p$, case 2 applies. We perform a left rotation, and the tree that results is shown in (c). Now, z is the left child of its parent, and case 3 applies. Recoloring and right rotation yield the tree in (d), which is a legal red-black tree.

Red-black tree – insertion – example

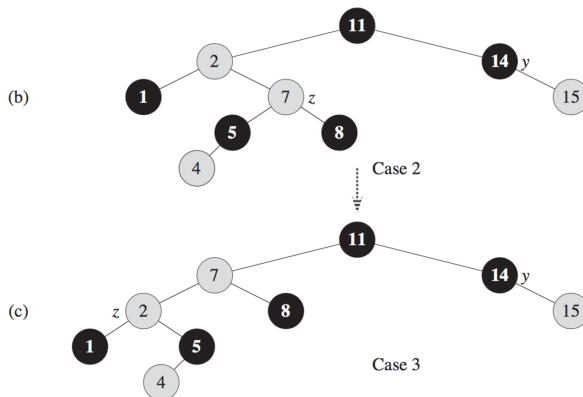


Figure 13.4 The operation of RB-INSERT-FIXUP. (a) A node z after insertion. Because both z and its parent $z.p$ are red, a violation of property 4 occurs. Since z 's uncle y is red, case 1 in the code applies. We recolor nodes and move the pointer z up the tree, resulting in the tree shown in (b). Once again, z and its parent are both red, but z 's uncle y is black. Since z is the right child of $z.p$, case 2 applies. We perform a left rotation, and the tree that results is shown in (c). Now, z is the left child of its parent, and case 3 applies. Recoloring and right rotation yield the tree in (d), which is a legal red-black tree.

Red-black tree – insertion – example

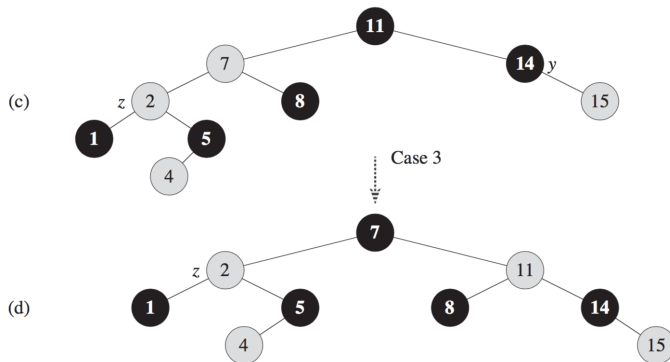
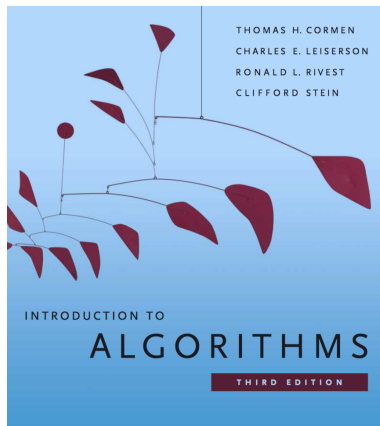


Figure 13.4 The operation of RB-INSERT-FIXUP. (a) A node z after insertion. Because both z and its parent $z.p$ are red, a violation of property 4 occurs. Since z 's uncle y is red, case 1 in the code applies. We recolor nodes and move the pointer z up the tree, resulting in the tree shown in (b). Once again, z and its parent are both red, but z 's uncle y is black. Since z is the right child of $z.p$, case 2 applies. We perform a left rotation, and the tree that results is shown in (c). Now, z is the left child of its parent, and case 3 applies. Recoloring and right rotation yield the tree in (d), which is a legal red-black tree.

References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.